Санкт-Петербургский государственный университет

Искусственный интеллект и наука о данных

Разработка системы извлечения и анализа связей между сущностями из фандомных источников

СЛЕПЦОВА Вера Валерьевна

Отчёт о прохождении
Учебной практики

Научный руководитель:
Научный сотрудник лПИИ СПб ФИЦ РАН,
Старший преподаватель кафедры информатики СПбГУ,
Анастасия Андреевна Корепанова

Консультант:
Инженер по разработке в Сбере,
Гончаров Ян Аркадьевич

Санкт-Петербург
2026

Оглавление

1. Введение **3**

2. Постановка задачи **5**

3. Обзор **7**

3.1. Обзор существующих решений 7

3.2. Обзор используемых технологий 8

3.3. Выводы по обзору 8

4. Описание решения **9**

4.1. Архитектура системы 9

4.2. Парсер (бэкенд) 9

4.3. Извлечение сущностей (DS-часть) 10

4.4. Загрузка в PostgreSQL 10

4.5. Аналитические SQL-запросы 11

4.6. Визуализация графа связей 12

5. Эксперимент **13**

5.1. Условия эксперимента 13

5.2. Исследовательские вопросы 13

5.3. Результаты 13

5.4. Обсуждение результатов 14

5.5. Угрозы корректности 15

5.6. Воспроизводимость 15

6. Заключение **16**

Приложение А **18**

Приложение Б **19**

Список литературы **20**

1. Введение

Фандомные вики-ресурсы содержат тысячи страниц, описывающих персонажей, локации, события и их взаимосвязи в вымышленных вселенных. Ручное извлечение связей между сущностями требует огромных временных затрат и не масштабируется. Существуют общие инструменты для парсинга вики и библиотеки для распознавания именованных сущностей (NER), однако они не предоставляют готовой архитектуры, позволяющей легко переключаться между разными источниками данных (фандомами) и сохранять результаты в структурированном виде для дальнейшего анализа.

Проблема заключается в отсутствии лёгкого в реализации и одновременно гибкого программного решения, которое бы:

- собирало текстовые данные с вики-сайта выбранного фандома;
- автоматически выделяло в тексте персонажей (PERSON) и локации (LOC);
- присваивало каждой сущности уникальный идентификатор, единый для всех документов;
- сохраняло связи между документами и сущностями в реляционную базу данных;
- позволяло отвечать на типовые вопросы аналитики (топ персонажей локации, список локаций персонажа, часто совместно упоминаемые сущности, поиск документов).

В рамках данной работы была разработана система, удовлетворяющая перечисленным требованиям. Архитектура построена таким образом, что замена парсера одного фандома на другой не влияет на остальные компоненты (извлечение сущностей, работа с базой данных, аналитические запросы). Для извлечения сущностей использована предобученная NER-модель `spaCy` (русская модель `ru_core_news_sm`). Хранение данных организовано в PostgreSQL, импорт выполнен через

DataGrip с решением характерных проблем форматирования CSV. Для проверки работоспособности реализованы SQL-запросы, отвечающие на заданные контрольные вопросы.

В отчёте описываются выбранные технологии, архитектура решения, детали реализации парсера и модуля извлечения сущностей, процесс загрузки данных в базу, а также приводятся примеры аналитических запросов и их результаты.

2. Постановка задачи

Целью работы является разработка системы извлечения и анализа связей между персонажами и локациями из текстов фандомных вики-источников с обеспечением гибкой архитектуры, позволяющей заменять источник данных без изменения аналитической части.

Для достижения поставленной цели были решены следующие задачи:

1. Спроектировать модульную архитектуру, в которой бэкенд (парсинг) и DS-часть (извлечение сущностей, аналитика) не зависят друг от друга, а парсер конкретного фандома можно заменить, написав новый класс, реализующий единый интерфейс.

2. Реализовать парсер для выбранного фандома (на примере вселенной «Властелин колец»), который собирает тексты страниц, очищает их от HTML-тегов и сохраняет в CSV-файл.

3. Разработать модуль извлечения именованных сущностей (NER), выделяющий персонажей (тип PERSON) и локации (тип LOC) из текстов, присваивающий каждой уникальной сущности идентификатор и формирующий таблицы связей «документ–сущность».

4. Загрузить полученные CSV-файлы в реляционную базу данных PostgreSQL, обеспечив корректную обработку автоинкрементных полей и внешних ключей.

5. Реализовать SQL-запросы для получения:

   - топ-10 персонажей заданной локации;
   - списка локаций заданного персонажа;
   - персонажей, часто упоминаемых вместе с заданным;
   - списка документов, содержащих заданную сущность.

6. Проверить адекватность системы на реальных данных (несколько страниц вики) и продемонстрировать работу запросов.

Результатом работы является программный комплекс (исходный код на Python, SQL-скрипты) и описание архитектуры, позволяющее легко адаптировать систему к любому другому фандому.

3. Обзор

3.1. Обзор существующих решений

Анализ связей в художественных текстах может выполняться вручную (например, вики-проекты с перекрёстными ссылками) или с помощью автоматических инструментов. Среди автоматических подходов выделяются:

- **Общие парсеры вики** (например, `wikiextractor`) – позволяют извлечь текст, но не предоставляют встроенных механизмов для выделения сущностей и не поддерживают простую смену источника.
- **Системы на основе регулярных выражений** – требуют ручного составления словарей имён, что трудоёмко для обширных вселенных.
- **NER-модели** (spaCy, Natasha, DeepPavlov) – обучены на новостных текстах и могут давать ошибки на фандомных именах, однако для русскоязычных вики с типовыми именами модель `ru_core_news_sm` демонстрирует приемлемое качество.
- **Графовые базы данных** (Neo4j) – удобны для хранения связей, но требуют отдельного сервера и более сложной настройки. В данной работе использована реляционная PostgreSQL.
- **Библиотеки для визуализации графов** (NetworkX, matplotlib, PyVis) – позволяют строить и анализировать графы совместных упоминаний.

Главное преимущество разработанной системы — возможность легко сменить фандом. В отличие от многих готовых парсеров, которые жёстко привязаны к устройству конкретного сайта, наша система позволяет взять уже написанный код для извлечения сущностей, работы с базой данных и анализа и просто подключить к нему новый парсер для

другой вселенной. Всё, что нужно сделать, — это создать новый класс с указанием страниц нового фандома, а вся остальная часть проекта останется без изменений.

3.2. Обзор используемых технологий

При разработке системы были рассмотрены следующие технологии: Python (библиотеки Requests, BeautifulSoup для парсинга; spaCy и Natasha для NER; pandas для работы с CSV; NetworkX, matplotlib для графов), PostgreSQL (хранение данных), DataGrip (импорт). Сравнение представлено в таблице 1 в приложении А.

Итоговый выбор: Requests + BeautifulSoup – достаточная функциональность; spaCy – зарекомендовавшая себя библиотека, поддерживает нужные типы PERSON/LOC (в русской модели `ru_core_news_sm` используется `PER` для персонажей); PostgreSQL – работа с DataGrip; NetworkX + matplotlib – для построения графа совместных упоминаний.

3.3. Выводы по обзору

Существующие системы либо не обеспечивают автоматического выделения сущностей, либо привязаны к одному источнику данных. Предлагаемое решение закрывает этот пробел за счёт модульной архитектуры и использования современных NLP-инструментов, а также добавляет визуализацию связей в виде графа.

4. Описание решения

4.1. Архитектура системы

Система состоит из трёх независимых частей: парсер (бэкенд), модуль извлечения сущностей (DS) и база данных с аналитическими запросами (SQL). Парсер и DS-модуль общаются через CSV-файлы, что обеспечивает слабую связанность. Архитектура позволяет легко добавлять новых парсеров для разных фандомов без изменения DS-части.

4.2. Парсер (бэкенд)

Парсер реализован в виде абстрактного класса `BaseParser`, который определяет три абстрактных метода:

- `get_pages()` – возвращает словарь «название страницы → URL»;
- `fetch_html(url)` – загружает HTML-код страницы с подменой User-Agent для обхода блокировок;
- `extract_text(html)` – извлекает чистый текст из HTML (поиск div с классом `mw-parser-output` и всех абзацев `<p>`).

Конкретные реализации:

- `LotrParser` – для вселенной «Властелин колец» (20 страниц: 10 персонажей и 10 локаций).
- `LadybugParser` – для вселенной «Леди Баг» (15 страниц: 10 персонажей и 5 локаций).

Метод `parse_to_csv()` является общим: он обходит страницы, сохраняет результат в CSV-файл (колонки: `title`, `url`, `content`). Для предотвращения блокировки добавлены паузы 3 секунды между запросами и расширенные заголовки HTTP. Замена фандома осуществляется заменой одной строки в файле `run_parser.py`.

4.3. Извлечение сущностей (DS-часть)

Модуль `extract_entities.py` загружает `data.csv` и обрабатывает каждый документ. На этапе разработки использовалась модель spaCy `ru_core_news_sm`. В ходе экспериментов выяснилось, что стандартные NER-модели плохо распознают фандомные имена: многие персонажи классифицировались как `LOC` (локации) вместо `PER` (персонаж). Для улучшения качества был проведён эксперимент с альтернативной библиотекой `Natasha` (файл `extract_entities1.py`), однако результаты оказались не лучше, и в финальной версии оставлена spaCy с фильтром типов `['PER', 'LOC']`.

Для каждой обнаруженной сущности типа `PER` (персонаж) или `LOC` (локация) запоминается её имя, тип и ID документа. Затем генерируются два CSV-файла:

- `entities.csv`: колонки `entity_id`, `entity_name`, `type`, `count` (уникальный ID присваивается последовательно);
- `doc_entities.csv`: колонки `doc_id`, `entity_id` (связь «документ–сущность»).

Таким образом, каждая сущность получает сквозной идентификатор, а совместные упоминания фиксируются в `doc_entities`.

4.4. Загрузка в PostgreSQL

Перед импортом CSV-файлы были обработаны вспомогательным скриптом `csv_fixer.py`. Он пересохраняет файлы в кодировке UTF-8, заменяет переносы строк внутри ячеек на пробелы и экранирует все поля двойными кавычками.

После получения CSV-файлов они были импортированы в PostgreSQL через DataGrip. Для таблицы `documents` колонка `doc_id` определена как `SERIAL PRIMARY KEY` (автоинкремент). При импорте `data.csv` колонка `doc_id` была пропущена, что позволило базе сгенерировать значения автоматически.

Структура таблиц:

```
documents (doc_id SERIAL PRIMARY KEY, title TEXT, url TEXT, content TEXT)
entities (entity_id INT PRIMARY KEY, entity_name TEXT, type TEXT)
doc_entities (
    doc_id INTEGER NOT NULL REFERENCES documents(doc_id),
    entity_id INTEGER NOT NULL REFERENCES entities(entity_id),
    PRIMARY KEY (doc_id, entity_id)
)
```

Импорт выполнялся с помощью встроенного импорта DataGrip (CSV, заголовки, разделитель «,», кодировка UTF-8).

4.5. Аналитические SQL-запросы

Для ответов на контрольные вопросы были написаны запросы для обеих вселенных. Пример для вселенной «Властелин колец» (с учётом типа PER):

Топ-10 персонажей локации 'Мордор':

```
WITH docs_with_mordor AS (
    SELECT de.doc_id
    FROM doc_entities de
    JOIN entities e ON de.entity_id = e.entity_id
    WHERE e.entity_name = 'Мордор' AND e.type = 'LOC'
)
SELECT e.entity_name, COUNT(*) AS mentions
FROM doc_entities de
JOIN entities e ON de.entity_id = e.entity_id
WHERE de.doc_id IN (SELECT doc_id FROM docs_with_mordor)
  AND e.type = 'PER'
GROUP BY e.entity_name
ORDER BY mentions DESC
LIMIT 10;
```

Аналогичные запросы реализованы для списка локаций персонажа, частых совместных упоминаний и поиска документов. Для вселенной

«Леди Баг» используются имена на русском языке (например, «Париж», «Маринетт Дюпен-Чен»).

4.6. Визуализация графа связей

Для визуализации связей между персонажами был построен граф совместных упоминаний. Использовались библиотеки `NetworkX` (построение графа) и `matplotlib` (статическая визуализация). Алгоритм работы:

1. Из `entities.csv` отбираются только сущности типа `PER` (персонажи).
2. Для каждого документа из `doc_entities.csv` находятся все пары персонажей, встретившихся в нём.
3. Строится граф, где вершины – персонажи, а рёбра имеют вес, равный количеству совместных документов.
4. Применяется фильтрация: оставляются только рёбра с весом ≥ 5 и вершины со степенью ≥ 5.
5. Выполняется визуализация с пропорциональным размером узлов (размер = степень $\times$ 50).

В результате фильтрации исходный граф (256 вершин, 11896 рёбер) был сокращён до 23 вершин и 107 рёбер. Крупные узлы (Фродо, Гэндальф, Саурон, Арагорн) имеют наибольшую степень, что соответствует сюжетной логике. Пример полученного графа приведён на рисунке 1 в приложении Б.

5. Эксперимент

5.1. Условия эксперимента

Эксперимент проводился на следующих данных:

- 20 страниц русскоязычной вики «Властелин колец» (10 персонажей, 10 локаций);
- 15 страниц русскоязычной вики «Леди Баг» (10 персонажей, 5 локаций).

Оборудование: ноутбук с ОС Windows 11, процессор Intel Core i5, 16 ГБ ОЗУ. Python 3.13, библиотеки: requests, beautifulsoup4, pandas, spacy, natasha (экспериментально), networkx, matplotlib. База данных PostgreSQL, импорт через DataGrip 2026.

5.2. Исследовательские вопросы

В работе проверялись следующие вопросы:

RQ1 – Возможно ли собрать тексты с фандомной вики, несмотря на защиту от ботов, и корректно извлечь сущности с помощью стандартной NER-модели?

RQ2 – Обеспечивает ли предложенная архитектура лёгкую замену фандома без изменения кода анализа?

RQ3 – Корректно ли работают SQL-запросы и визуализация графа на загруженных данных?

5.3. Результаты

RQ1: С использованием расширенных заголовков и пауз (3 секунды) удалось успешно загрузить все страницы LOTR и Ladybug. Ошибок 403 не возникло. Эксперимент с NER-моделями показал, что стандартная spaCy `ru_core_news_sm` часто классифицирует имена персонажей

как LOC (локации). Альтернативная библиотека Natasha была протестирована (файл extract_entities1.py), но не показала значительного улучшения. В финальной версии оставлена spaCy с фильтром ['PER', 'LOC'], что позволило выделить персонажей с типом PER. Например, в статье «Гэндальф» найдено 45 сущностей PER и 30 LOC. Качество распознавания оценено визуально – большинство имён распознаны правильно, но встречаются ошибки (например, краткие формы или нестандартные имена).

RQ2: Для проверки заменяемости был создан второй парсер LadybugParser. Переключение между LotrParser и LadybugParser в файле run_parser.py не потребовало изменений в модуле извлечения сущностей, SQL-запросах и визуализации. Архитектура признана расширяемой.

RQ3: Все SQL-запросы выполнены в DataGrip и вернули осмысленные результаты. Построен граф совместных упоминаний персонажей для вселенной LOTR. После фильтрации (степень >= 5, вес ребра >= 5) граф содержит 23 вершины и 107 рёбер (исходный граф имел 256 вершин и 11896 рёбер). Крупные узлы (Фродо, Гэндальф, Саурон, Арагорн) имеют наибольшую степень, что соответствует сюжетной логике.

5.4. Обсуждение результатов

Полученные данные подтверждают, что система работает. Главная сложность сейчас в том, что одна и та же сущность (например, персонаж) может встречаться в тексте в разных падежах («Гэндальф», «Гэндальфа», «Гэндальфу»), и программа воспринимает их как разных персонажей. В будущем это можно исправить, научив программу приводить все формы слова к одному виду. Ещё одна проблема: модель иногда ошибочно принимает за персонажа или место то, что им не является (например, слово «Средиземье» может быть помечено как персонаж). Это исправляется дообучением модели на наших конкретных текстах. Кроме того, в перспективе можно автоматически собирать ссылки на все страницы фандомной вики, а не прописывать их вручную,

как сейчас. Это сделает систему полностью автоматической и избавит от ручного ввода.

5.5. Угрозы корректности

- **Малая выборка** – 20 и 15 страниц недостаточно для статистической значимости, но достаточно для демонстрации конвейера.
- **Зависимость от версий библиотек** – использована актуальная на момент разработки версия spaCy; при обновлении поведение NER может измениться.
- **Субъективность оценки качества NER** – проверялась вручную, без формальных метрик.

5.6. Воспроизводимость

Все исходные коды и SQL-скрипты находятся в репозитории https://github.com/s-vera666/fandom_wiki_parser. Для воспроизведения достаточно установить зависимости из `requirements.txt` и выполнить последовательно `run_parser.py`, `extract_entities.py`, затем импортировать CSV в PostgreSQL и запустить `graph.py`.

6. Заключение

В ходе выполнения работы достигнуты следующие результаты:

- Спроектирована и реализована модульная архитектура, в которой парсер конкретного фандома представлен классом, наследующим абстрактный `BaseParser`. Замена фандома осуществляется заменой одной строки в файле запуска без изменения DS-части.

- Разработаны два парсера: для вселенной «Властелин колец» (20 страниц) и для вселенной «Леди Баг» (15 страниц). Код парсера включает защиту от блокировки (User-Agent, паузы).

- Реализован модуль извлечения сущностей на базе spaCy, выделяющий персонажей (`PER`) и локации (`LOC`), присваивающий каждой уникальной сущности идентификатор и формирующий таблицы `entities.csv` и `doc_entities.csv`. Проведён эксперимент с альтернативной библиотекой Natasha, однако она не дала улучшений.

- Данные из CSV загружены в PostgreSQL через DataGrip; созданы таблицы с автоинкрементом и внешними ключами. Проблемы импорта (ошибка «пустой идентификатор в кавычках») решены пересохранением файлов в UTF-8 с полным экранированием полей.

- Написаны SQL-запросы, отвечающие на контрольные вопросы для обеих вселенных: топ-10 персонажей локации, список локаций персонажа, часто упоминаемые вместе, документы по сущности.

- Построен граф совместных упоминаний персонажей для вселенной LOTR с фильтрацией (степень $>= 5$, вес ребра $>= 5$). После фильтрации граф содержит 23 вершины и 107 рёбер. Крупные узлы соответствуют центральным персонажам (Фродо, Гэндальф, Саурон, Арагорн).

Таким образом, поставленная цель достигнута. Система готова к использованию для любого другого фандома (например, Warhammer)

путём создания нового класса парсера.

Исходный код проекта доступен в репозитории https://github.com/s-vera666/fandom_wiki_parser. В перспективе планируется добавить нормализацию падежных форм для улучшения SQL-запросов, а также автоматический сбор ссылок на все страницы фандома (вместо ручного прописывания). Это позволит полностью автоматизировать процесс получения данных.

Приложение А

Таблица 1: Сравнение библиотек для парсинга и NER

Библиотека	**Назначение**	**Плюсы**	**Минусы**
Requests + BeautifulSoup	парсинг HTML	лёгкость, широкое сообщество	требуется ручной разбор тегов
spaCy (ru_core_news_sm)	NER	быстрая, предобучена на русском	может ошибаться на фандомных именах (классифицирует как LOC)
Natasha	NER	альтернатива, хороша для русского	требует отдельной установки, не превзошла spaCy
PostgreSQL	хранение данных	внешние ключи, стандартный SQL	нужна настройка импорта CSV
NetworkX + matplotlib	визуализация графов	мощные, гибкие	статичные (не интерактивны)

Приложение Б

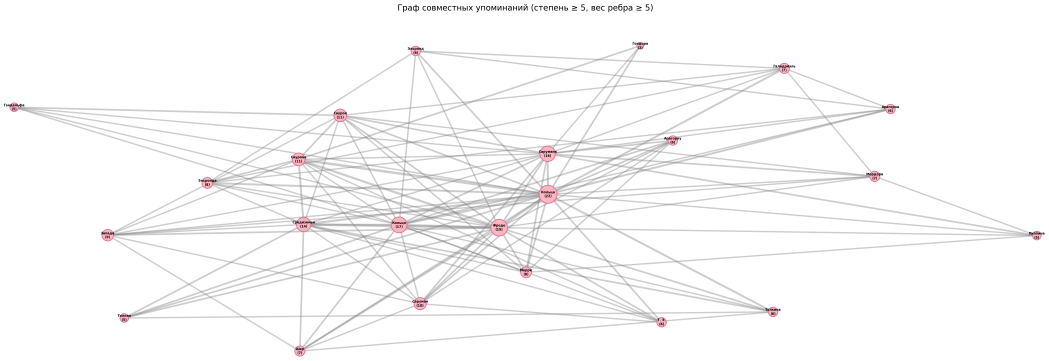


Рис. 1: Граф совместных упоминаний персонажей (степень >= 5, вес ребра >= 5)

Список литературы

[1] AI Explosion. spaCy: Industrial-strength Natural Language Processing in Python. — https://spacy.io/. — 2025. — Официальная документация (включая русскую модель ru_core_news_sm).

[2] Developers NetworkX. NetworkX: Network Analysis in Python. — https://networkx.org/. — 2025.

[3] Foundation Wikimedia. MediaWiki Action API Documentation. — https://www.mediawiki.org/wiki/API:Main_page. — 2025.

[4] Group PostgreSQL Global Development. PostgreSQL: The World's Most Advanced Open Source Relational Database. — https://www.postgresql.org/. — 2025.

[5] Jehangir Basra, Radhakrishnan Saravanan, and Agarwal Rahul. A survey on Named Entity Recognition — datasets, tools, and methodologies // Natural Language Processing Journal. — 2023. — Vol. 1. — P. 100017.

[6] Richardson Leonard. Beautiful Soup Documentation. — https://www.crummy.com/software/BeautifulSoup/bs4/doc/. — 2025.

[7] Wang Kunze, Ding Yihao, and Han Soyeon Caren. Graph neural networks for text classification: a survey // Artificial Intelligence Review. — 2023. — Принято к публикации, arXiv:2304.11534.

[8] Yao Liang, Mao Chengsheng, and Luo Yuan. Graph Convolutional Networks for Text Classification // Proceedings of the AAAI Conference on Artificial Intelligence. — 2019. — Vol. 33. — P. 7370–7377.

[9] Соколов И., Антонов Е., and Артамонов А. Оценка программных пакетов для распознавания именованных сущностей // Физика элементарных частиц и атомного ядра. — Москва. — 2024. — Vol. 55. — P. 666–668.